



SC1006

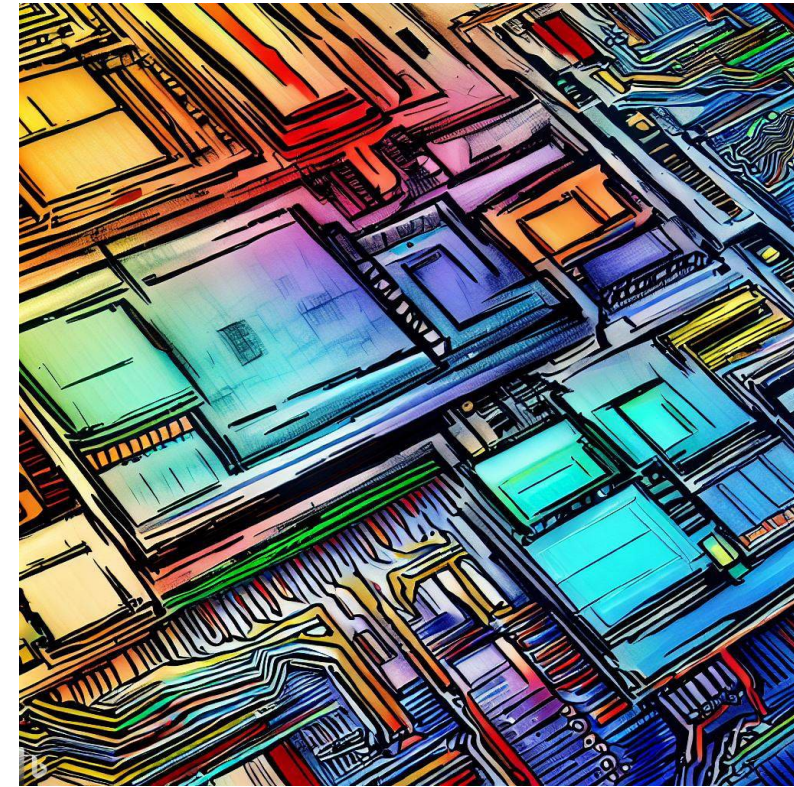
Computer Organisation and Architecture

Direct Memory Access

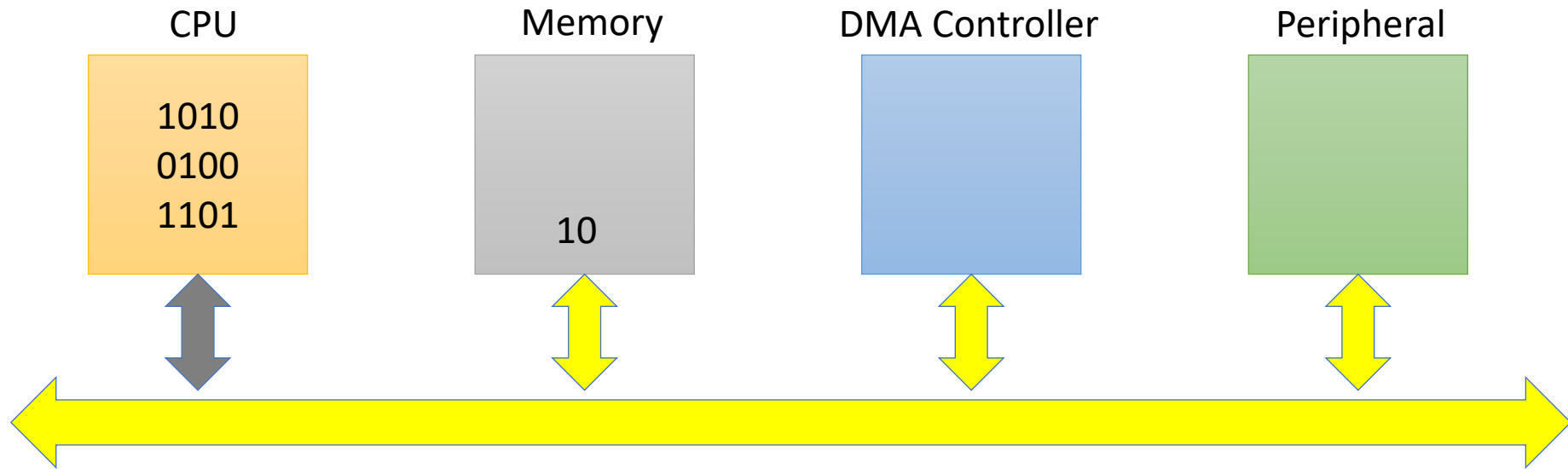
Prof Lin WeiSi
College of Computing and Data Science, NTU
Email: wslin@ntu.edu.sg

Optimizing CPU resources in Data Transfer

- Till now, we have been using **CPU** to perform the **data transfer**.
- This becomes increasing **inefficient** as amount of data increase as CPU has to spend most of its time moving data.
- As such, CPU has **less time** to perform algorithm processing.
- DMA controller (DMAC) is added to **relieve CPU** of the data transfer task.
- DMAC has **dedicated hardware** that could **move data more efficiently** than CPU in scenarios where **complex address manipulation** is required. E.g. de-interleaving Left and Right Channel **Audio data**.
- **If there are no conflicts in hardware resources** used, it is possible for data transfer via DMA and CPU execution to occur **simultaneously**.
- If there is a **conflict in hardware resources used**, e.g. both DMAC and the CPU needs the system bus, then access will be given to the one with **higher priority**.
- Who has the higher **priority** and whether the priority is configurable depends on processor design and is thus **processor specific**.

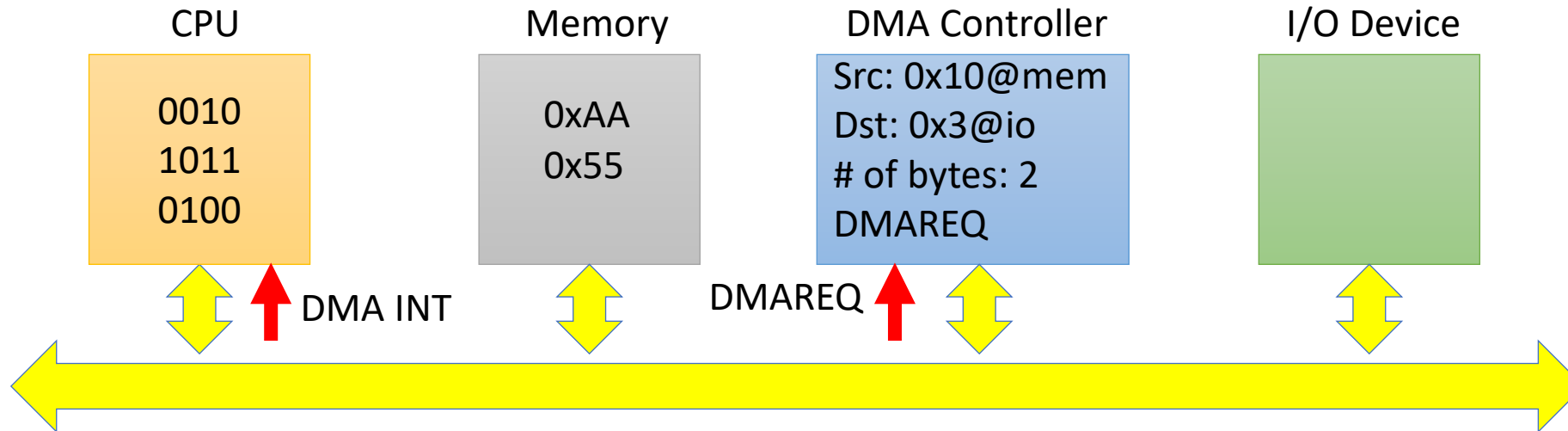


DMA Controller (DMAC)



- DMAC is a **Data Bus Controller** module that performs data transfer independent of CPU, between **Memory and memory, I/O Peripheral and I/O peripheral, Memory and I/O Peripheral**.
- Generates address and initiates read/write operation between devices mentioned above.
- Note that 'Peripheral' here could be **internal or external** peripherals
- The DMAC shown here uses the **Fetch and Deposit** DMA mechanism where the data goes into the internal buffer of the DMAC before being transferred to the Destination.

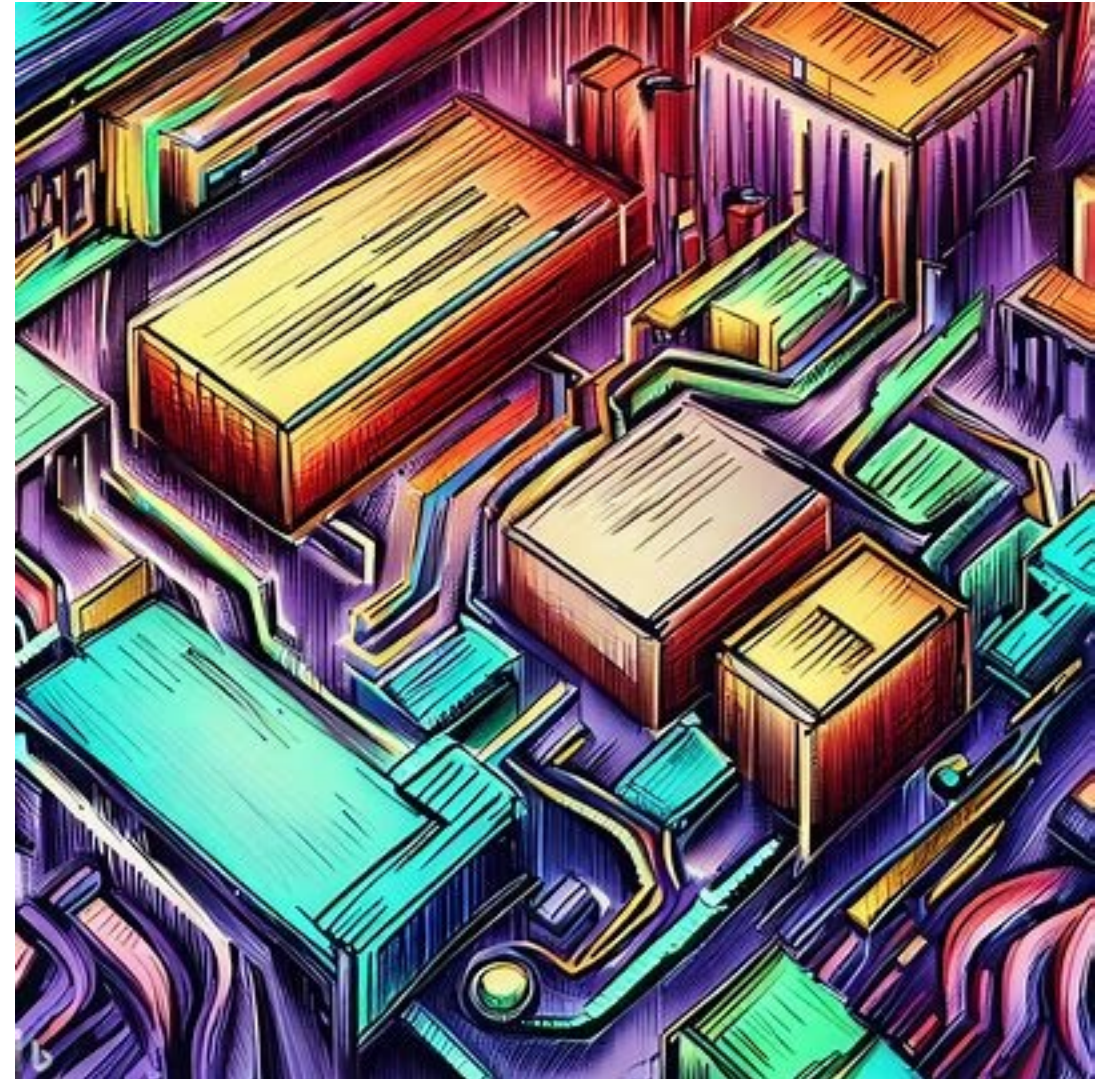
Basic DMA Process



- DMA **Configuration** Parameters initialization (by CPU to DMAC)
 - Source Address
 - Destination Address
 - Amount of data to transfer
 - DMA Trigger signal (DMAREQ, interrupt, etc)
- CPU proceed with its own task while **DMAC wait** for DMA **Trigger** Signal.
- Once DMA **Trigger** occurs, DMAC **request** to take over system bus.
- DMAC transfer data according to configuration.
- DMAC **notify CPU of data completion** (typically via interrupts) and **release** system bus.

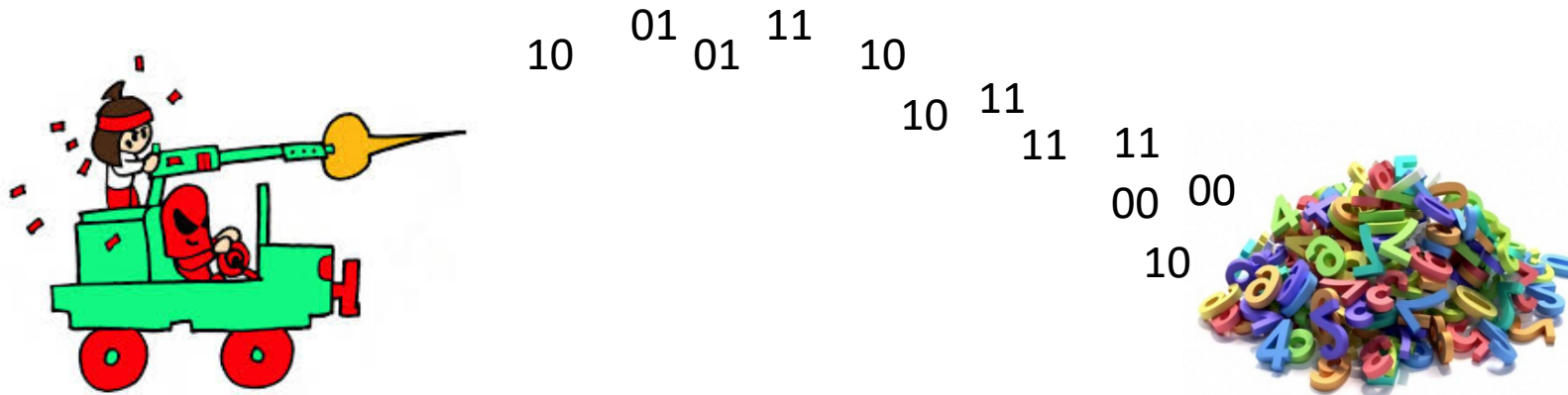
DMA Mode of Operation

- Different processors has **different DMAC design**, typically with slight variance in terms of how they transfer data and the data type (Audio, Video etc) they are optimized for.
- In general, DMAC could transfer the block of data they are tasked to transfer in the following mode
 - **Burst**
 - **Cycle Stealing**
 - **Transparent**
- Do note that you may encounter DMAC design that varies from the basic mode of operation discussed here but general concept will still apply.
- In fact, for majority of the time, you'll encounter DMAC design that uses **a combination of the modes** described



Burst Mode

- The DMA controller gains control of the system bus and **transfer multiple units of data** before returning control of the bus to the CPU.
- Note that the burst could be **the entire block of data** DMAC is tasked to transfer, or a **subset of the block**, i.e. it may take a few burst to complete the transfer of the entire block.
- CPU may continue to operate as long as it does not need access the particular system bus that DMAC is using.
- If CPU needs to access the system bus, **CPU may be suspended** till DMAC has completed its data transfer.
- Fast data transfer rate but may render the CPU inactive for **longer period** of time.



Cycle Stealing

- DMAC releases the system bus after transferring **one unit of data**.
- Depending on processor design, DMAC tends to execute the data transfer
 - **Between CPU instructions**
 - **Between Pipeline stages**
- CPU may be suspended if it need to access the system bus but **suspend time is shorter** as only one unit is transferred at one time.
- Transfer rate is **slower** than in Burst Mode but will CPU will only be **inactive for very short period of time**.
- **Favoured** in application which requires CPU to be **responsive** e.g. real-time security status monitoring.



Between
CPU Instr

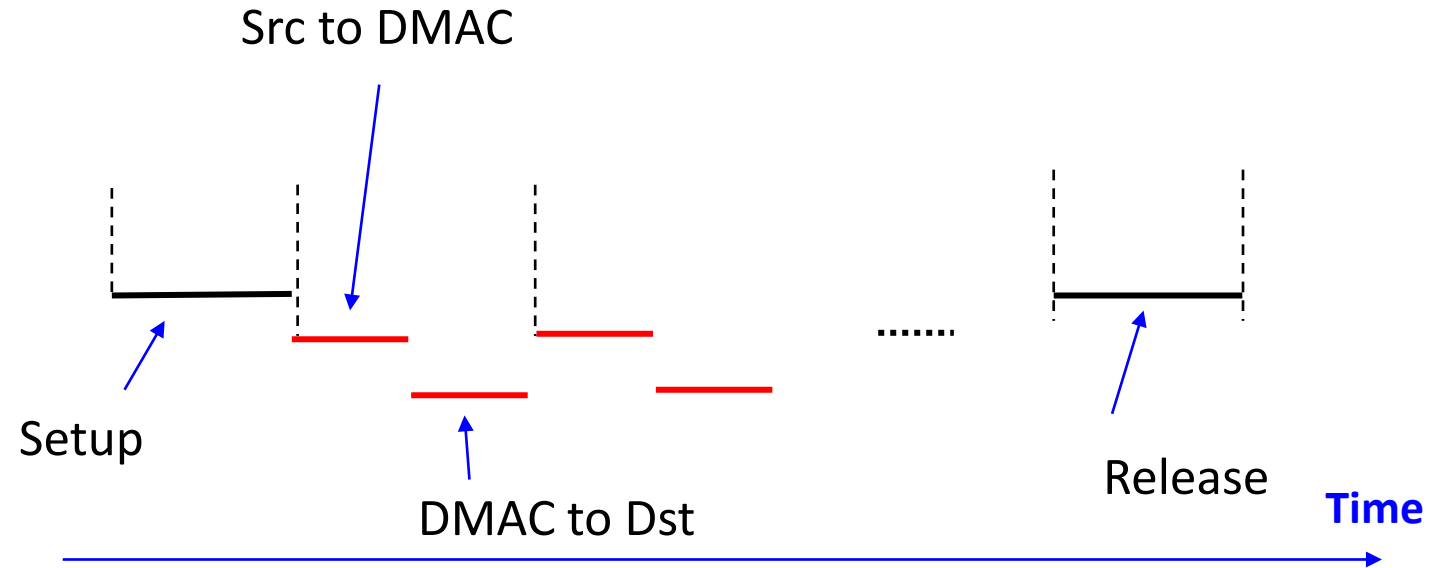
CPU	Instr1	Instr2		Instr3		Instr4
DMAC			Data		Data	

Between
Pipeline
Stages

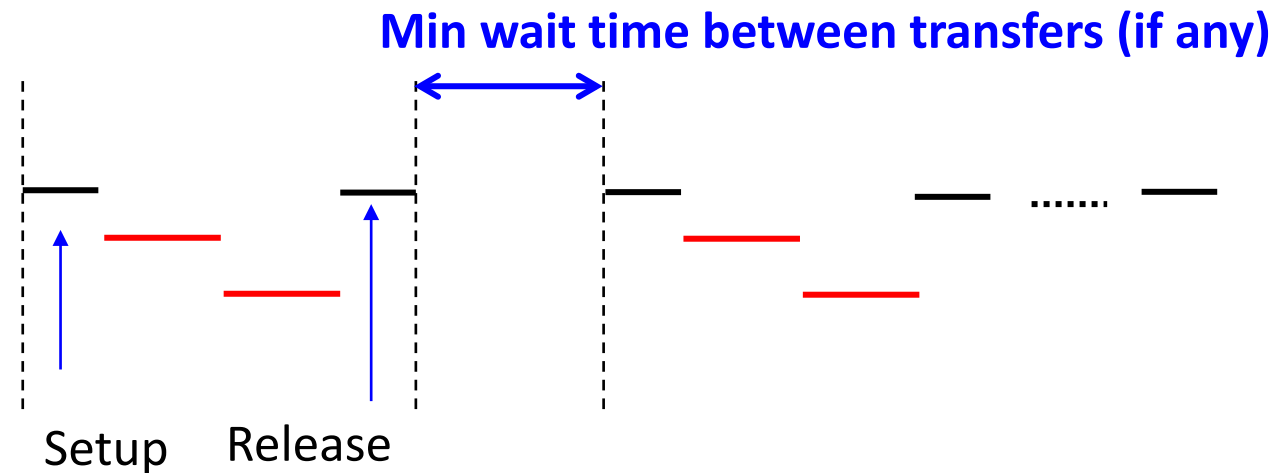
CPU	Fetch Instr	Decode Instr		Fetch Operand	Exe Instr	
DMAC			Data			Data

Fetch and Deposit (Timings)

Burst



Cycle Stealing



Transparent Mode



- DMAC transfer data only when **CPU is not using the data bus**.
- **Zero impact** to CPU performance in terms of data bus access.
- Potentially the slowest transfer rate among the three modes.
- **More Complex hardware** needed to detect when CPU is not using the data bus.
- The CPU activity detection technique could also be used by some processors as a cue to trigger a burst transfer only when CPU is not using the system bus.

Comparison of DMA Transfer Modes

- **Burst**

- Allow **faster** data transfer
- CPU may be **suspended** for **longer** period of time
- May not be suitable for Real-time applications with CPU
- Suitable for application where transfer is bursty e.g. HDD file transfer.

- **Cycle Stealing**

- **Interleaving** DMA data transfer with CPU instructions allows CPU to continue executing its program while DMAC is doing data transfer.
- **CPU performance lower** due to interleaving of DMA transfers, but would be **more responsive** than in Burst mode.
- **Slower data transfer rate** than Burst mode.
- Suitable for Real-time applications with CPU

- **Transparent**

- Potentially would not affect CPU performance at all.
- **Slowest data transfer rate** but best CPU response
- **More complex hardware** needed to detect when CPU is not using the bus.

